МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ

по лабораторной работе №3
по дисциплине «Интеллектуальные методы анализа данных»
Тема: Реализация метода кластеризации данных с использованием природных временных рядов

Студент гр. 3381 Кулаков А.Ю.
Преподаватель Мандрикова Б.С.

Санкт-Петербург
2026

Цель работы.

Реализация метода c-средних с использованием наблюдений нейтронных мониторов.

Задание.

1. Скачать данные из источника https://www.nmdb.eu/nest/ за периоды и станции, указанные по вариантам в Приложении 1 (таблицы п.1 и п.2). Использовать данные с одноминутным временным разрешением.

2. При наличии пропусков в данных заполнить их медианным значением (функция median в MatLab).

3. Сформировать матрицу входных данных data из преобразованных временных рядов в фиксированное число признаков (например, среднее значение, стандартное отклонение, амплитуда и др.).

4. Реализовать метод нечёткой кластеризации c-средних. В MatLab воспользоваться функцией [centers,U] = fcm(data,Nc,options), где data — входные данные, Nc — количество кластеров, options — параметры метода.

5. Выполнить проверку работоспособности построенного метода кластеризации.

6. Сделать соответствующие выводы, оформить отчёт по работе.

Выполнение работы.

Работа выполнялась на языке Python. Для обработки данных использовались библиотеки pandas и numpy, для стандартизации — StandardScaler, для нечёткой кластеризации — функция cmeans библиотеки skfuzzy. Исходный код программы приведён в приложении А.

Для варианта 10 использованы данные станции Апатиты (APTY). Наблюдения спокойного состояния и магнитной бури считываются соответственно из файлов calm.csv и storm.csv с разделителем «;».

```
data_calm = pd.read_csv('./calm.csv', sep=';')
data_storm = pd.read_csv('./storm.csv', sep=';')
```

В функции fill_med() столбец start_date_time приводится к формату даты и времени, устанавливается индекс, удаляются дублирующиеся временные метки. Для каждого имеющегося дня создаётся минутная сетка из 1440 отсчётов. Пропуски в RCORR_E заполняются медианой соответствующего дня.

```
data['RCORR_E'] = data.groupby(data.index.date)['RCORR_E'].transform(
    lambda x: x.fillna(x.median())
)
```

В функции feature_engineering() по суточным временным рядам вычисляются четыре признака: среднее значение (mean), дисперсия (var), амплитуда (amplitude) и 80-й процентиль (q80). Амплитуда рассчитывается как разность максимального и минимального значений. Результаты двух классов объединяются в таблицу data.

```
features = data.groupby(data.index.date)['RCORR_E'].agg(['mean', 'var', 'min', 'max',
                                                         ('q80', lambda x: x.quantile(0.8))])
features['amplitude'] = features['max'] - features['min']
```

Для спокойных дней задаётся class = 1, для дней магнитной бури — class = 2. Проверяется отсутствие пропусков в рассчитанных признаках. Первые пять строк матрицы, напечатанные программой:

```
         date        mean       var  amplitude       q80  class
0  2018-01-03  179.239129  6.233944     17.209  181.3674      1
1  2018-01-05  179.341115  6.823091     17.900  181.4984      1
2  2018-01-06  179.861391  6.264880     18.107  181.9178      1
3  2018-01-07  180.630853  5.495911     18.013  182.5092      1
4  2018-01-10  178.696110  7.549737     21.979  180.9044      1
```

Размер таблицы, выведенный программой:

```
(89, 6)
```

Для кластеризации используются четыре столбца признаков. Перед применением метода с-средних они стандартизируются функцией StandardScaler().

```
X = data[['mean', 'var', 'amplitude', 'q80']].to_numpy()
scaler = StandardScaler()
X = scaler.fit_transform(X)
```

Нечёткая кластеризация выполняется методом cmeans с числом кластеров c = 2, показателем нечёткости m = 2, точностью error = 0.005, максимальным числом итераций 1000 и seed = 42.

```
centers, u, _, _, _, _, fpc = fuzz.cluster.cmeans(
    X.T, c=2, m=2, error=0.005, maxiter=1000, seed=42
)
```

Для каждого наблюдения выбирается кластер с наибольшей степенью принадлежности u. При необходимости номера кластеров меняются местами в соответствии с проверкой доли совпадений с исходными классами. Полученный номер записывается в столбец cluster.

```
clusters = u.argmax(axis=0)
real = data['class'].to_numpy() - 1
if np.mean(clusters == real) < np.mean(1 - clusters == real):
    clusters = 1 - clusters
data['cluster'] = clusters + 1
```

Центры кластеров после обратного преобразования StandardScaler, выведенные программой в исходном порядке алгоритма:

```
[[182.68323512   6.8695537   18.93102988 184.86850732]
 [179.70752769   7.13406918  19.72973601 181.91808184]]
```

Коэффициент разбиения, выведенный программой:

```
0.7009477157981633
```

Для проверки работоспособности строится таблица распределения исходных классов по полученным кластерам:

```
cluster   1   2
class
1        20  24
2        16  29
```

В результате выполнения программы получена доля совпадений:

```
0.550561797752809
```

Результаты вместе с датами, признаками, исходным классом и найденным кластером сохраняются в файл clusters.csv.

Выводы.

В ходе работы из суточных рядов наблюдений нейтронного монитора APTY сформированы четыре признака: среднее значение, дисперсия, амплитуда и 80-й процентиль. По полученным 89 наблюдениям выполнена нечёткая кластеризация с-средних на два кластера. Коэффициент разбиения составил 0.7009477157981633. По таблице распределения исходных классов и кластеров доля совпадений составила 0.550561797752809 (55,06%). Таким образом, метод кластеризации выполнен и проверен, однако совпадение с заданным разделением на спокойные дни и дни магнитной бури невысоко.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

```
import pandas as pd
import numpy as np
import skfuzzy as fuzz
from sklearn.preprocessing import StandardScaler

def fill_med(data):
    data['start_date_time'] = pd.to_datetime(data['start_date_time'])
    data = data.set_index('start_date_time').sort_index()
    data = data[~data.index.duplicated()]

    full_range = pd.DatetimeIndex([])
    for day in data.index.normalize().unique():
        full_range = full_range.append(pd.date_range(day, periods=1440, freq='1min'))

    data = data.reindex(full_range)
    data['RCORR_E'] = data.groupby(data.index.date)['RCORR_E'].transform(
        lambda x: x.fillna(x.median())
    )
    return data

def feature_engineering(data):
    features = data.groupby(data.index.date)['RCORR_E'].agg(['mean', 'var', 'min', 'max',
                                                             ('q80', lambda x: x.quantile(0.8))])
    features['amplitude'] = features['max'] - features['min']
    return features[['mean', 'var', 'amplitude', 'q80']].rename_axis('date').reset_index()

data_calm = pd.read_csv('./calm.csv', sep=';')
data_storm = pd.read_csv('./storm.csv', sep=';')

data_calm = fill_med(data_calm)
data_storm = fill_med(data_storm)

features_calm = feature_engineering(data_calm)
features_storm = feature_engineering(data_storm)

features_calm['class'] = 1
features_storm['class'] = 2

data = pd.concat([features_calm, features_storm], ignore_index=True)

if data[['mean', 'var', 'amplitude', 'q80']].isna().any().any():
    raise ValueError('Не хватает данных для некоторых дат')

print(data.head())
print(data.shape)

X = data[['mean', 'var', 'amplitude', 'q80']].to_numpy()

scaler = StandardScaler()
X = scaler.fit_transform(X)

centers, u, _, _, _, _, fpc = fuzz.cluster.cmeans(
    X.T, c=2, m=2, error=0.005, maxiter=1000, seed=42
)

clusters = u.argmax(axis=0)
real = data['class'].to_numpy() - 1

if np.mean(clusters == real) < np.mean(1 - clusters == real):
    clusters = 1 - clusters

data['cluster'] = clusters + 1

print('Центры кластеров:')
print(scaler.inverse_transform(centers))

print('Коэффициент разбиения:', fpc)
print(pd.crosstab(data['class'], data['cluster']))
print('Доля совпадений:', np.mean(data['class'] == data['cluster']))

data.to_csv('./clusters.csv', sep=';', index=False)
```